

Streamlit + Azure SDK Implementation Plan

Project Overview

Transform manual backoffice user management into a streamlined web application using Streamlit and Azure SDK for Python.

Phase 1: Foundation Setup (Weeks 1-2)

1.1 Environment Setup

Day 1-2: Development Environment

```
bash

# Create virtual environment
python -m venv venv
source venv/bin/activate # Linux/Mac
# or
venv\Scripts\activate # Windows

# Install core dependencies
pip install streamlit
pip install azure-keyvault-secrets
pip install azure-identity
pip install azure-mgmt-containerinstance
pip install pandas
pip install python-dotenv
pip install streamlit-authenticator
```

Day 3-4: Azure Resources Setup

- Create Azure App Service or Container Instance for hosting
- Configure Managed Identity for the hosting service
- Set up Azure Key Vault access policies
- Create Azure AD App Registration for authentication

Day 5-7: Project Structure

```

backoffice-user-manager/
├── app.py                # Main Streamlit application
├── requirements.txt       # Python dependencies
├── .streamlit/
│   └── config.toml       # Streamlit configuration
├── src/
│   ├── __init__.py
│   ├── azure_services/
│   │   ├── __init__.py
│   │   ├── keyvault_client.py # Key Vault operations
│   │   ├── auth_manager.py   # Authentication handling
│   │   └── audit_logger.py    # Audit logging
│   ├── models/
│   │   ├── __init__.py
│   │   └── user_model.py     # User data models
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── csv_handler.py    # CSV processing
│   │   └── validators.py     # Input validation
│   └── pages/
│       ├── __init__.py
│       ├── user_management.py # User CRUD operations
│       ├── bulk_import.py    # Bulk user import
│       └── audit_logs.py     # Audit trail view
├── tests/
│   ├── __init__.py
│   ├── test_keyvault.py
│   └── test_user_operations.py
└── deployment/
    ├── Dockerfile
    ├── docker-compose.yml
    └── azure-pipelines.yml

```

1.2 Authentication Setup

Day 8-10: Azure AD Integration

- Configure Azure AD authentication
- Set up role-based access control
- Create user groups for different permission levels

Phase 2: Core Development (Weeks 3-4)

2.1 Azure SDK Integration

Key Vault Client Implementation

python

```
# src/azure_services/keyvault_client.py
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential
import pandas as pd
import io

class KeyVaultUserManager:
    def __init__(self, vault_url: str, secret_name: str):
        self.credential = DefaultAzureCredential()
        self.client = SecretClient(vault_url=vault_url, credential=self.credential)
        self.secret_name = secret_name

    def get_users_df(self) -> pd.DataFrame:
        """Retrieve users CSV from Key Vault and return as DataFrame"""

    def update_users_df(self, df: pd.DataFrame) -> bool:
        """Update users CSV in Key Vault from DataFrame"""

    def backup_users(self) -> str:
        """Create backup of current users data"""
```

User Data Model

python

```
# src/models/user_model.py
from dataclasses import dataclass
from typing import Optional, List
from datetime import datetime

@dataclass
class BackofficeUser:
    username: str
    email: str
    full_name: str
    role: str
    department: str
    is_active: bool
    created_date: datetime
    last_modified: datetime
    permissions: List[str]

    def to_dict(self) -> dict:
        """Convert to dictionary for CSV serialization"""

    @classmethod
    def from_dict(cls, data: dict) -> 'BackofficeUser':
        """Create from dictionary (CSV deserialization)"""
```

2.2 Streamlit UI Development

Main Application Structure

python

app.py

```
import streamlit as st
from src.azure_services.auth_manager import authenticate_user
from src.pages import user_management, bulk_import, audit_logs
```

```
def main():
    st.set_page_config(
        page_title="Backoffice User Manager",
        page_icon="👤",
        layout="wide"
    )

    # Authentication
    if not authenticate_user():
        st.stop()

    # Navigation
    pages = {
        "User Management": user_management,
        "Bulk Import": bulk_import,
        "Audit Logs": audit_logs
    }

    # Sidebar navigation
    selected_page = st.sidebar.selectbox("Navigate", list(pages.keys()))
    pages[selected_page].render()

if __name__ == "__main__":
    main()
```

User Management Page

python

```
# src/pages/user_management.py
import streamlit as st
import pandas as pd
from src.azure_services.keyvault_client import KeyVaultUserManager
from src.models.user_model import BackofficeUser

def render():
    st.title("👤 User Management")

    # Initialize Key Vault client
    kv_manager = KeyVaultUserManager(
        vault_url=st.secrets["KEYVAULT_URL"],
        secret_name=st.secrets["USERS_SECRET_NAME"]
    )

    # Tabs for different operations
    tab1, tab2, tab3 = st.tabs(["View Users", "Add User", "Edit User"])

    with tab1:
        render_users_table(kv_manager)

    with tab2:
        render_add_user_form(kv_manager)

    with tab3:
        render_edit_user_form(kv_manager)

def render_users_table(kv_manager):
    """Display users in a table with search and filter options"""

def render_add_user_form(kv_manager):
    """Form for adding new users"""

def render_edit_user_form(kv_manager):
    """Form for editing existing users"""
```

Phase 3: Advanced Features (Weeks 5-6)

3.1 Role-Based Access Control

Permission Matrix

python

```
# src/models/permissions.py
ROLE_PERMISSIONS = {
    "Super Admin": [
        "create_user", "edit_user", "delete_user", "view_users",
        "bulk_import", "view_audit_logs", "manage_roles"
    ],
    "HR Admin": [
        "create_user", "edit_user", "view_users", "bulk_import"
    ],
    "Department Manager": [
        "view_users", "edit_user_department"
    ],
    "Viewer": [
        "view_users"
    ]
}
```

3.2 Audit Logging

Audit Logger Implementation

python

```
# src/azure_services/audit_logger.py
import logging
from datetime import datetime
from typing import Dict, Any

class AuditLogger:
    def __init__(self, log_analytics_workspace_id: str):
        self.workspace_id = log_analytics_workspace_id

    def log_user_action(self,
                       user_id: str,
                       action: str,
                       target_user: str = None,
                       details: Dict[str, Any] = None):
        """Log user management actions for audit trail"""

    def get_audit_logs(self,
                       start_date: datetime,
                       end_date: datetime,
                       user_filter: str = None) -> pd.DataFrame:
        """Retrieve audit logs for display"""
```

3.3 Data Validation and Error Handling

Input Validation

python

```
# src/utils/validators.py
```

```
import re
```

```
from typing import List, Tuple
```

```
def validate_email(email: str) -> Tuple[bool, str]:  
    """Validate email format"""
```

```
def validate_username(username: str, existing_users: List[str]) -> Tuple[bool, str]:  
    """Validate username uniqueness and format"""
```

```
def validate_user_data(user_data: dict) -> Tuple[bool, List[str]]:  
    """Comprehensive user data validation"""
```

Phase 4: Deployment (Weeks 7-8)

4.1 Containerization

Dockerfile

dockerfile

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8501
```

```
HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health
```

```
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

4.2 Azure Deployment Options

Option A: Azure Container Instances

yaml

```
# deployment/aci-deployment.yml
apiVersion: 2019-12-01
location: eastus
name: backoffice-user-manager
properties:
  containers:
    - name: streamlit-app
      properties:
        image: your-registry/backoffice-user-manager:latest
        ports:
          - port: 8501
            protocol: TCP
        resources:
          requests:
            cpu: 1
            memoryInGB: 2
        environmentVariables:
          - name: KEYVAULT_URL
            value: https://your-keyvault.vault.azure.net/
```

Option B: Azure App Service

yaml

```
# deployment/app-service-config.yml
kind: linux
sku: B1
runtime: python|3.9
startup_command: streamlit run app.py --server.port=8000 --server.address=0.0.0.0
```

4.3 CI/CD Pipeline

Azure DevOps Pipeline

azure-pipelines.yml

trigger:

branches:

include:

- main

pool:

vmImage: ubuntu-latest

variables:

containerRegistry: 'your-registry.azurecr.io'

imageName: 'backoffice-user-manager'

stages:

- stage: Build

jobs:

- job: BuildAndTest

steps:

- task: UsePythonVersion@0

inputs:

versionSpec: '3.9'

- script: |

pip install -r requirements.txt

python -m pytest tests/

displayName: 'Run Tests'

- task: Docker@2

inputs:

command: 'buildAndPush'

repository: \$(imageName)

dockerfile: 'Dockerfile'

containerRegistry: \$(containerRegistry)

tags: |

\$(Build.BuildId)

latest

- stage: Deploy

jobs:

- deployment: DeployToAzure

environment: 'production'

strategy:

runOnce:

deploy:

steps:

- task: AzureContainerInstances@0

inputs:

azureSubscription: 'your-service-connection'

resourceGroupName: 'your-resource-group'

location: 'East US'

containerGroupName: 'backoffice-user-manager'

containerImage: '\$(containerRegistry)/\$(imageName):\$(Build.BuildId)'

Phase 5: Testing and Validation (Week 9-10)

5.1 Testing Strategy

Unit Tests

python

tests/test_user_operations.py

import pytest

from src.models.user_model import BackofficeUser

from src.utils.validators import validate_email, validate_username

def test_user_creation():

"""Test user model creation and validation"""

def test_email_validation():

"""Test email validation logic"""

def test_csv_operations():

"""Test CSV import/export functionality"""

Integration Tests

python

tests/test_keyvault_integration.py

import pytest

from unittest.mock import Mock, patch

from src.azure_services.keyvault_client import KeyVaultUserManager

@patch('src.azure_services.keyvault_client.SecretClient')

def test_keyvault_user_retrieval(mock_secret_client):

"""Test Key Vault integration"""

5.2 Load Testing

Streamlit Performance Testing

python

```
# tests/load_test.py
import concurrent.futures
import requests
import time

def test_concurrent_users():
    """Test application under concurrent user load"""
```

Configuration Management

Environment Variables

bash

```
# .env (for development)
KEYVAULT_URL=https://your-keyvault.vault.azure.net/
USERS_SECRET_NAME=backoffice-users-csv
AZURE_CLIENT_ID=your-app-registration-id
AZURE_TENANT_ID=your-tenant-id
LOG_ANALYTICS_WORKSPACE_ID=your-workspace-id
```

Streamlit Secrets

toml

```
# .streamlit/secrets.toml
[azure]
keyvault_url = "https://your-keyvault.vault.azure.net/"
users_secret_name = "backoffice-users-csv"
tenant_id = "your-tenant-id"
client_id = "your-app-registration-id"

[app]
admin_users = ["admin@company.com", "hr@company.com"]
```

Success Metrics and Monitoring

Key Performance Indicators

- User management task completion time: < 2 minutes
- Application uptime: > 99.5%
- User adoption rate: > 80% within 30 days
- Error rate: < 1% of operations

- Audit compliance: 100% of actions logged

Monitoring Setup

python

```
# src/utils/monitoring.py
```

```
import logging
```

```
from azure.monitor.opentelemetry import configure_azure_monitor
```

```
def setup_monitoring():
```

```
    """Configure Application Insights monitoring"""
```

```
    configure_azure_monitor(
```

```
        connection_string=os.getenv("APPLICATIONINSIGHTS_CONNECTION_STRING")
```

```
    )
```

Risk Mitigation

Data Backup Strategy

- Automated daily backups of user data
- Version control for CSV changes
- Rollback capability for failed updates

Security Measures

- Azure AD integration for authentication
- Managed Identity for Azure service access
- Role-based access control
- Input validation and sanitization
- Audit logging for all operations

Disaster Recovery

- Multi-region deployment capability
- Backup restoration procedures
- Failover testing schedule

Cost Estimation

Monthly Costs (Estimated)

- Azure Container Instances (B1): \$15-30
- Azure Key Vault operations: \$5-10

- Azure AD Premium (if needed): \$6/user/month
- Log Analytics: \$5-15
- **Total estimated cost: \$30-60/month**

This plan provides a comprehensive roadmap for implementing your Streamlit-based user management solution with clear deliverables, timelines, and technical specifications.